CMP-142 Object Oriented Programming

Lab-07

Topics:

- Inheritance
- Modes Of Inheritance
- File Handling

Instructions!

1. This is a **graded lab,** you are strictly **NOT** allowed to discuss your solutions with your fellow colleagues, even not allowed to ask how he/she is doing, it may result in negative marking. You can **ONLY** discuss with your TAs or instructor.
2. Indent your code properly.
3. Use meaningful variable and function names. **Use the camelCase notation.**
4. Save your work frequently. Make a habit of pressing CTRL+S after every line of code you write.
5. **Do NOT use any global or static variable(s).** However, global named constants may be used.
6. Make sure that there are **NO dangling pointers or memory leaks** in your programs.
7. **Make sure that all functions have been implemented and can be called in the main function. Otherwise, no marks will be awarded for that function.**
8. **<u>You have time till 4:45 pm</u>**

[Task 01] Military Time to Standard Time Conversion

Create a program that demonstrates the conversion of time from military (24-hour) format to standard (12-hour) format using a base `Time` class and a derived `MilTime` class. The program should enforce input validation for hours and seconds, ensuring proper time values are entered.

1. **Base Class: `Time`**

Private Members:

`hours`: Holds the standard hour in 12-hour format. `minutes`: Holds the minutes. `seconds`: Holds the seconds

Public Member Functions:

`setStandardTime(int, int, int)`: Sets the standard time values.
`displayStandardTime()`: Displays the time in the standard 12-hour format (e.g., 02:30 PM).

2. Derived Class: MilTime

Design a class called MilTime that is derived from the Time class (inherited). This inheritance is of protected mode.

The MilTime class should convert time in military (24-hour) format to the standard time format used by the Time class.

Private Members:

`milHours:` Contains the hour in 24-hour format. For example, 1:00 pm would be stored as 1300 hours, and 4:30 pm would be stored as 1630 hours.
`milSeconds:` Contains the seconds in standard format.

Public Member Functions:

`Constructor:` The constructor should accept arguments for the hour and seconds, in military format. The time should then be converted to standard time and stored in the hours, min, and sec variables of the Time class.
`setTime:` Accepts arguments to be stored in the milHours and milSeconds variables. The time should then be converted to standard time and stored in the hours, min, and sec variables of the Time class.
`getHour:` Returns the hour in military format.
`getStandHr:` Returns the hour in standard format.

Demonstrate the class in a program that asks the user to enter the time in military format. The program should then display the time in both military and standard format.

Input Validation: ***The MilTime class should not accept hours greater than 2359, or less than 0. It should not accept seconds greater than 59 or less than 0. Any sort of error must be handled gracefully.***

[Task 02] Demonstrating Modes of Inheritance in a Bank System

Design a banking system that includes three types of accounts (`BankAccount`, `SavingsAccount`, and `FixedDepositAccount`) and an independent `Admin` class. The inheritance hierarchy should demonstrate **public**, **protected**, and **private** inheritance. The functionality of each type of inheritance must be visibly different to highlight their unique properties.

1. **Base Class: `Person`**

Private Member Variables:

```
string name,
int age,
string nationalID
```

Protected Member:

```
string phoneNumber.
```

Public Member & Functions:

```
string address.
displayPersonalInfo();
```
to display public and protected details.

2. **Intermediate Base Class: `BankAccount`**

This class represents a generic bank account. By generic we mean, any sort of bank account. This class is inherited from Person using "**public inheritance**".

Protected Members:

```
string accountNumber,
double balance,
double minimumBalance.
```

Public Methods:

`deposit()`: Adds to the balance.
`withdraw()`: Deducts from the balance if conditions are met.
`displayAccountDetails()`: Displays account information and "displays personal information".

3. Derived Class 1: `SavingsAccount`

This class is inherited from **BankAccount** using **"protected inheritance"**.

Private Members:

```
double interestRate
```

Public Methods:

`calculateInterest()` Use this formula for calculating interest rate:
balance * interestRate * years / 100

Also you need to do one more thing, Restrict access to `deposit()` and `withdraw()` from outside the class.

4. Derived Class 2: `FixedDepositAccount`

This class is also inherited from `BankAccount` using private inheritance.

Private Members:

```
double depositAmount
 int lockPeriod
```

Public Methods:

`createFD()` to set deposit details.
Display details through a dedicated method.

5. Admin Class:

This class does not require any inheritance. It only perform the following operations:

Access details of a `SavingsAccount`
Access details of a `FixedDepositAccount`

Main Function:

You need to create objects of `SavingsAccount` and `FixedDepositAccount`. And then demonstrate the following tasks:

- Public inheritance: Direct access to `BankAccount` members via `SavingsAccount`.
- Protected inheritance: Restricted access to `BankAccount` members in `SavingsAccount` but accessible via `Admin`-like methods.
- Private inheritance: `BankAccount` members fully encapsulated in `FixedDepositAccount`.

[Task 03] File Handling with Matrix Class and Overloaded Multiplication

---

Create a C++ program that defines a `Matrix` class, allowing the user to input two matrices. The program should store the first matrix in a file named `mat1.txt`, the second matrix in a file named `mat2.txt`, and the result of their multiplication in a file named `mat1xmat2.txt`. The program should overload the multiplication operator (`*`) to perform matrix multiplication, with the result being saved to the `mat1xmat2.txt` file.

But before performing multiplication, you first need to read the matrices from their respective files. The program should also include input validation to ensure the matrices can be multiplied (i.e., the number of columns of the first matrix must equal the number of rows of the second matrix). The matrices should be read from the files and displayed accordingly.

Following is the definition of Matrix class:

NOTE: ***No function prototype will be provided. You have to do this on your own. Moreover, make sure that no dangling pointer or inaccessible memory errors are found.***

Private Members:

```
int rows, cols;
int** data;   allocates 2D array dynamically
```

Public Member Functions:

- Constructor to initialize matrix with dimensions
- Destructor
- Function to take matrix input from user
- A const function to display the matrix
- Overloaded * operator to multiply two matrices
- Function to save matrix to a file
- Function to read matrix from a file